第二章　迭代—分形图形

本章内容



§2.1 迭代与分形

§2.2 复变函数迭代

§2.3 分形与分形维



教学目标

1. 训练编程能力。
2. 学习有关分形的知识。
3. 掌握编程要求。

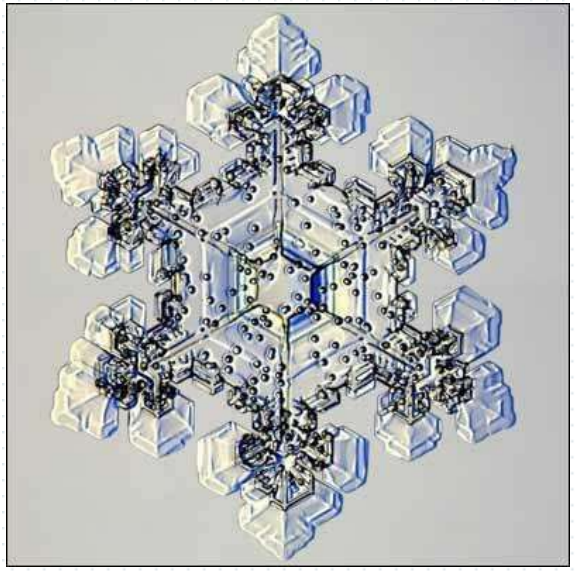
①正确性 ：模型正确，计算方法正确
②计算速度快：避免不必要或重复的计算，
采用省时的算法
③语句简洁规范：可读性强，通用性强

程序演示（树，三角形，雪花…）









程序从何而来?

	分形树图	分形三角形图	分形雪花图
0. 实数描点法	1	0	0
1. 复数描点法	2	1	3
2. 插入新点法	2	1	3
3. 相似移动法	1	2	3

§ 2.1 迭代与分形

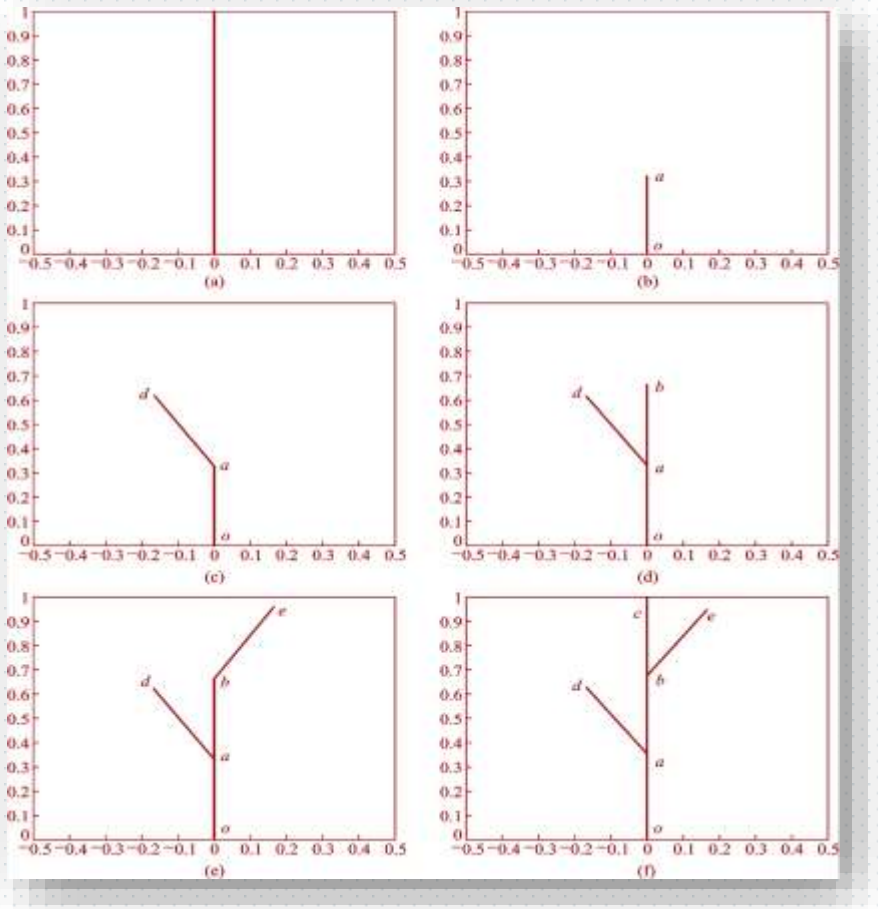
§2.1.1 分形树

将一条线段三等分

分点上各画一条长度为原线段三分之一的线段

新线段与原线段成固定的夹角（如π/3）

得到树枝状的图形



§ 2.1.1 分形树

1.树程序1

```
u =[0, 0; 0, 1];                                  %线段oc端点的坐标
subplot(3,3,1),
plot(u(:,1),u(:,2)),                              %画oc
axis([-0.5,0.5,0,1])
theta=pi/6;                                       %夹角是下面用到参数
for n=1:8                                         %画分形树的循环次数
    uuu=[];
for I=0:(length(u)/2-1)
                                  %用上次分形树的线段画下一分形树
    p1=(u(2*I+1,:)*2+u(2*I+2,:))/3 ;    % 线段的1/3处如a点
    p2=(u(2*I+1,:)+u(2*I+2,:)*2)/3;     % 线段的2/3处如b点
lp=[cos(theta),-sin(theta);sin(theta),cos(theta)]*...
```

注：用《程序界面》演示

```
[(u(2*l+2,1)-u(2*l+1,1));u(2*l+2,2)-u(2*l+1,2)]/3 ;
lp=p1+lp';                                  % 向左转动的点如d点
rp=[cos(theta),sin(theta);-sin(theta),cos(theta)]*...
     [(u(2*l+2,1)-u(2*l+1,1));u(2*l+2,2)-u(2*l+1,2)]/3;
rp=p2+rp';                                  % 向右转动的点如e点
uu=[u(2*l+1,:);p1;p1;lp;p1;p2;p2;rp;p2;u(2*l+2,:)];
                                                %将点排序
uuu=[uuu;uu];                          %将各线段所得数据排列起来
End
u=[uuu];
subplot(3,3,n+1),plot(u(:,1),u(:,2)),                    %画新图
axis([-0.5,0.5,0,1])
end
```

程序说明

- 两重循环，分别计算点坐标与重复画图次数。
- 计算每条线段两端点坐标，用plot连线。
- 线段端点坐标(x_1, y_1)。
- 线段的1/3处的坐标$\left(\frac{2x_1+x_2}{3}, \frac{2y_1+y_2}{3}\right)$
- 线段的2/3处的坐标$\left(\frac{x_1+2x_2}{3}, \frac{y_1+2y_2}{3}\right)$
- 逆时针转动以后的新坐标$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$

北京师范大学

端点排序与连线方法

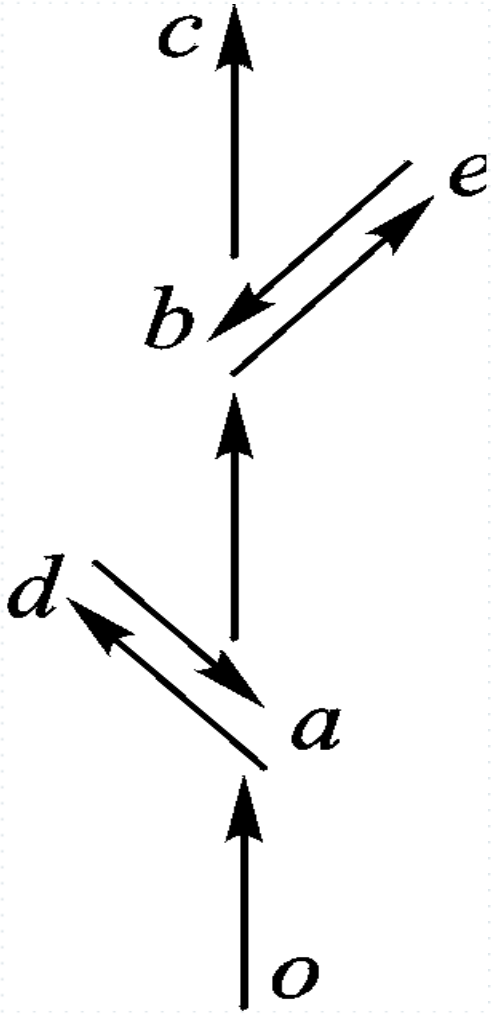
只有奇数连线在下次分型中有效，只计算奇数连线

第一次：画线段oc

第二次：画线段
oa，ad，ab，be，bc

产生端点o，a，d，b，e，c

实际画出线段oa，aa，ad，
da，ab，bb，be，eb，bc



2.树程序2

改进一

- 点的坐标
- 线段长度
- 坐标转动θ

效果：运算快

改进二

- 端点坐标：
  用矩阵存储

效果：少一重循环

评价：作图方法相同，计算过程略快，
程序更简洁，可读性更强。

```
u=[0,i]; subplot(3,3,1);  plot(u), axis([-0.5,0.5,0,1])
for n=1:7
uuu=[];
for I=0:length(u)/2-1
diff=(u(2*I+2)-u(2*I+1))/3;   %在第一次循环是oa的长度
p1=u(2*I+1)+diff;             %在第一次循环是计算a点坐标
p2=p1+diff;                   %在第一次循环是计算b点坐标
```

（续前一页）

```
lp=p1+diff*(sqrt(3)/2+1/2*i);
                                %在第一次循环是计算d点坐标
rp=p2+diff*(sqrt(3)/2-1/2*i);
                                %在第一次循环是计算e点坐标
uu=[u(2*l+1);p1;p1;lp;p1; p2;p2;rp;p2;u(2*l+2)];
uuu=[uuu;uu];
end
u=uuu;
subplot(3,3,n+1);plot(u), axis([-0.5,0.5,0,1])
end
```

3.树程序3

改进： 不计算已知的点坐标，将新计算点插入已知点之间

```
new=[0,i];  subplot(3,3,1);
plot(new);  axis([-0.5,0.5,0,1 ]);
for k=1:8
old=new;
n=length(old)/2-1;                          %上次生成的线段数
diff=(old(2:2:end)-old(1:2:end-1))/3;  %每条线段1/3的长度
p1=old(1:2:end-1)+diff;
p2=p1+diff;
lp=p1+diff*(sqrt(3)/2+1/2*i);
rp=p2+diff*(sqrt(3)/2-1/2*i);
```

%以下的计算在第一次循环中计算10个端点o，a，a，d，a，b，b，e，b，c的坐标

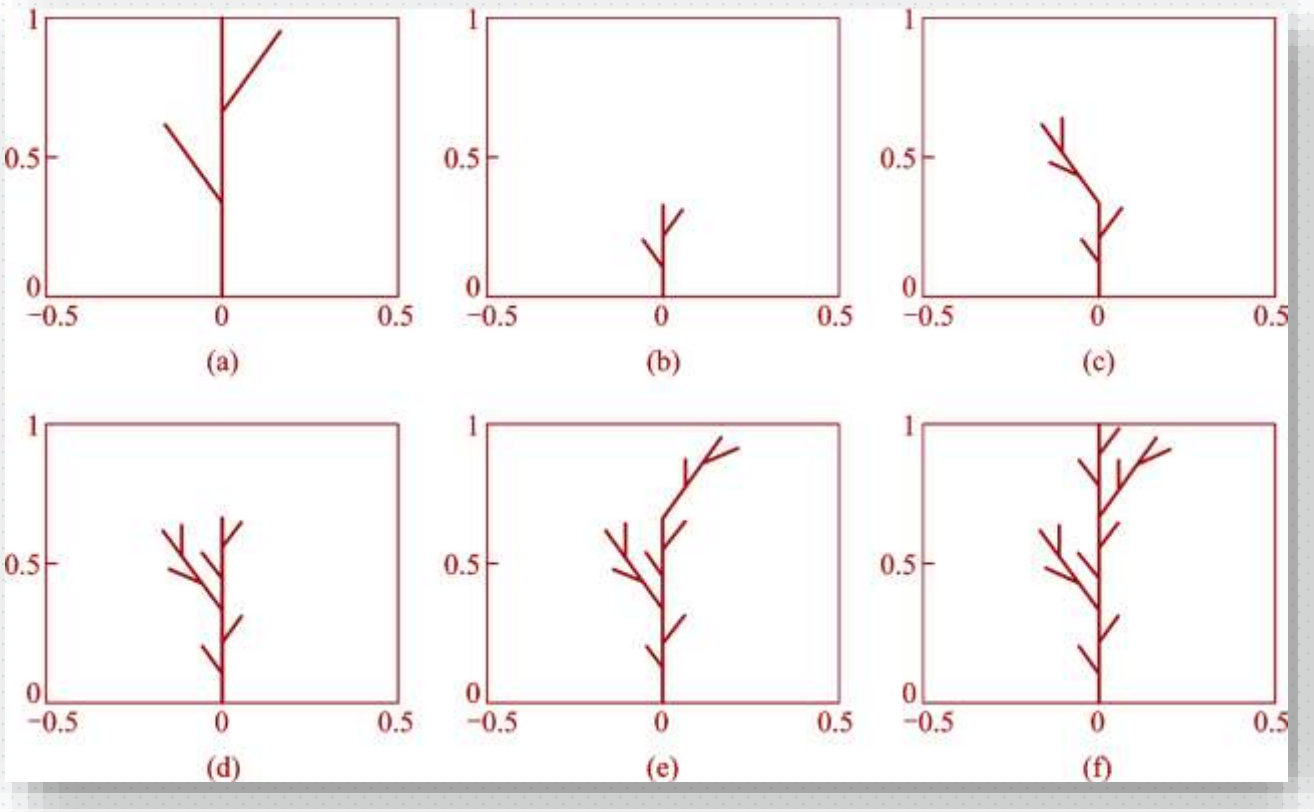
```
new(1:10:10*n+1)=old(1:2:end-1)
                                %各次循环分别计算1,11,21....点
new(2:10:10*n+2)=p1;  %各次循环分别算第2,12,22,...点
new(3:10:10*n+3)=p1;  %各次循环分别算第3,13,33,...点
new(4:10:10*n+4)=pl;
new(5:10:10*n+5)=p1;
new(6:10:10*n+6)=p2;
new(7:10:10*n+7)=p2;
```

```
%以下的计算在第一次循环中计算10个端点o，a，a，d，a，b，b，e，b，c的坐标
   new(8:10:10*n+8)=rp;
   new(9:10:10*n+9)=p2;
   new(10:10:10*(n+1))=old(2:2:end); %第10,20,...点
   subplot(3,3,k+1),
   plot(new)
   axis([-0.5,0.5,0,1 ]);
end
```

```
u=[0,i];   subplot(3,3,1);     plot(u),     axis([-0.5,0.5,0,1])
for k=2:8
m=u/3;                                          %将图形缩小到1/3
uu=[m, ...                                      %放置缩小后的图形m
i/3+m*(sqrt(3)*0.5+0.5i),...
                                %将m在虚轴上向上移动1/3后向左旋转
m+i/3,...                                       %将m在虚轴上向上移动1/3
2i/3+m*(sqrt(3)*0.5-0.5i),...
                                %将m在虚轴上向上移动2/3后向右旋转
m+2i/3];                                        %将m在虚轴上向上移动2/3
subplot(3,3,k);          plot(uu),     axis([-0.5,0.5,0,1])
u=uu;
end
```

4.树程序4

将前一个分形图缩小到原来的1/3 → 摆放到5个不同的位置 → 形成后一个分形图形



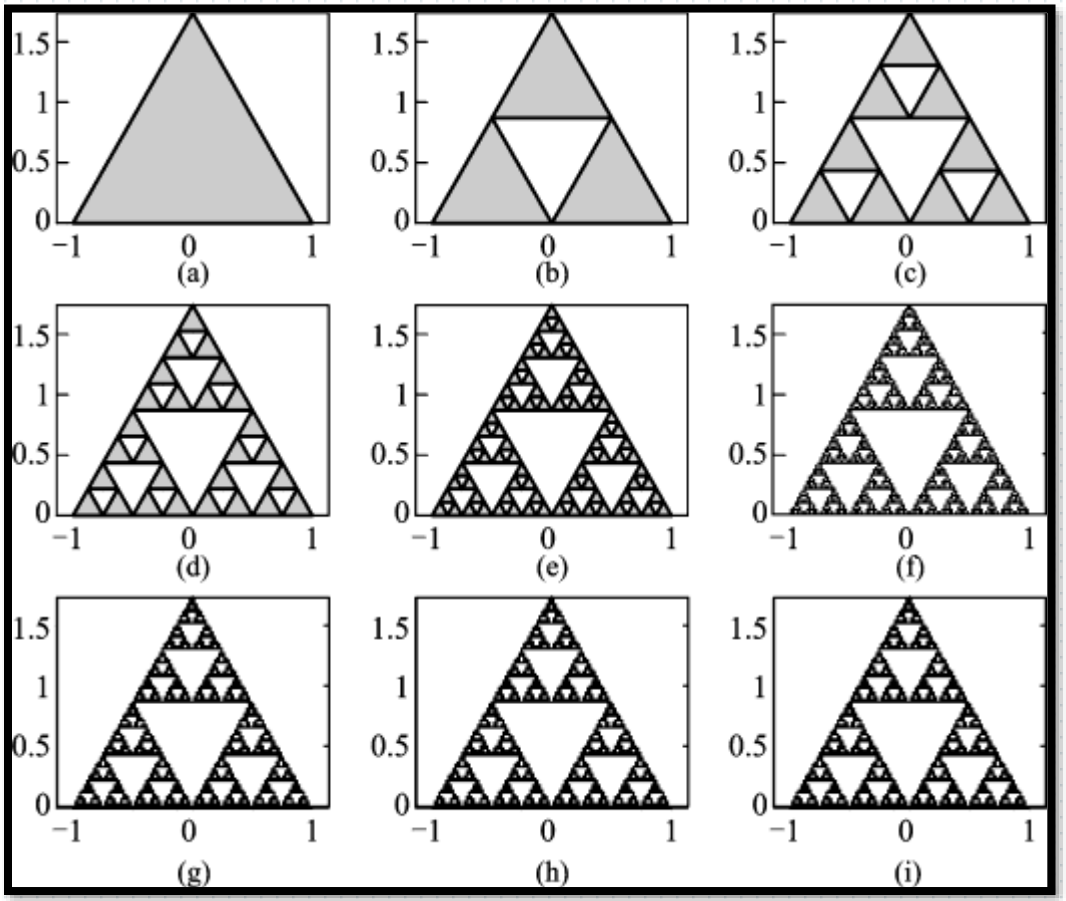
利用分形图形的自相似性来画图

§2.1.2 Sierpinski 三角形

将三角形四等分，得四个较小三角形。

舍去中间小三角形，保留有颜色的三个。

按上述分割与舍去法操作下去。



§ 2.1.2 Sierpinski triangle

1.分形三角形程序1

```
for m=0:8                                          %最大的三角形等分的次数
subplot(3,3,m+1)
    f=[-1;  sqrt(3)*i;   1];
%用3×1的矩阵表示原始三角形的顶点坐标
old=f;                                             从最初三角形开始计算所有三角形数据
    for k=1:((3^m-1)/2);
%将上次循环的一个小三角形分为三个三角形
new=[old(1),     (old(1)+old(2))/2,  (old(1)+old(3))/2;...
        (old(1)+old(2))/2, old(2),          (old(2)+old(3))/2;...
        (old(1)+old(3))/2, (old(2)+old(3))/2,    old(3)];
```

（接下页）

（接上页）

```
f=[f,new] ;          %将新数据加入原矩阵后，矩阵增加3列
old=f(:,k+1) ;
                %下一步取第k+1列元素(即第k+1个三角形)计算
End
 ff=f(:,((3^m-1)/2+1):((3^(m+1)-1)/2)) ;
%前期三角形不用，仅对最后生成的小三角数据加以填色
fill(real(ff), imag(ff),'g');
axis equal;
end
```

2.分形三角形程序2

```
new=[-1;  sqrt(3)*i;  1];
subplot(3,3,1);
fill(real(new),imag(new),'g');
axis equal;
for m=1:8
old=new;                          %调用上次循环所得的数据
a=(old(1,:)+old(2,:))/2;          % 3种新的不同的数据
b=(old(1,:)+old(3,:))/2;
c=(old(2,:)+old(3,:))/2;
N=numel(new)/3;                   % 计算原矩阵的列数
```

（接下页）

（接上页）

```
  new(1, 1:3:3*N)=old(1,:); %将新的数据插入形成新的矩阵
  new(1, 2:3:3*N+1)=a;
  new(1, 3:3:3*N+2)=b;
  new(2, 1:3:3*N)=a;
  new(2, 2:3:3*N+1)=old(2,:);
  new(2, 3:3:3*N+2)=c;
  new(3, 1:3:3*N)=b;
  new(3, 2:3:3*N+1)=c;
  new(3, 3:3:3*N+2)=old(3,:);
  subplot(3,3,m+1);                                %画图填色
  fill(real(new),imag(new),'g');
  axis equal;
end
```

3.分形三角形程序3

```
u=[0,1,0.5+0.5i*sqrt(3),0];                    %原始的三角形
subplot(3,3,1)
fill(real(u), imag(u),'g')
for k=1:8
m=0.5*u;                                       %将数据缩小到1/2
u=[m(1:end-k-1), ...                           %将移动形成的3个三角形的
m + 0.5,...                                    %数据按画图要求排列
m + 0.25+0.25*i*sqrt(3), 0];
subplot(3,3,k+1)
fill(real(u), imag(u),‘g’)          改变数据结构导致问题
end
```

4.分形三角形程序4
——（递归调用法编迭代程序）

```
function sjx(A1,A2,A3,N1)                    %函数文件的固定格式
triangle1=[A1,A2,A3,A1];                     %原始的大三角形
B1=(A1+A2)/2;  B2=(A1+A3)/2; B3=(A2+A3)/2;
                                             %三角形的3个中点
triangle2=[B1,B2,B3,B1];                     %由3个中点连接的三角形
N2=N1-1;                                     %完成了一次迭代
if N2>0;                                     %继续新的迭代
```

（接下页）

（接上页）

```
sjx(A1,B1,B2,N2);
                    %递归调用后每次迭代又成3个新的迭代
sjx(B1,A2,B3,N2);
sjx(B2,B3,A3,N2);
subplot(2,3,N2)
fill([real(triangle1)],[imag(triangle1)],'g');
                                %将大三角形填成绿色
hold on
fill([real(triangle2)],[imag(triangle2)],‘w’)
                                %将中间小三角形填白色
end
```

5.用递归调用程序来计算阶乘

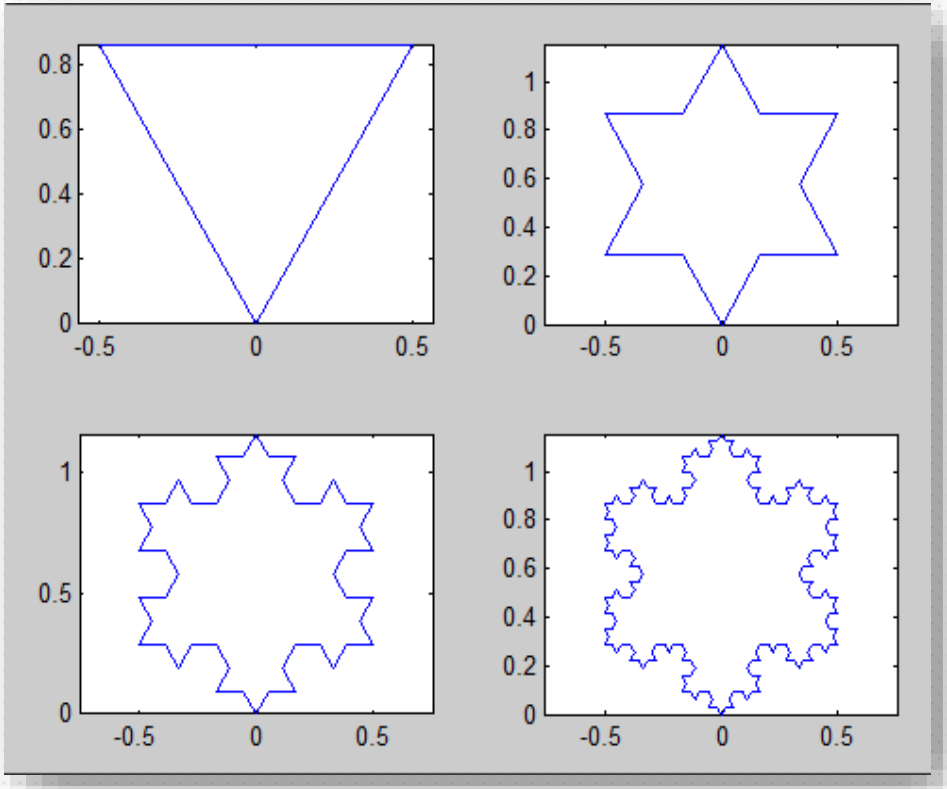
```
%program fact.m
function output=fact(n)
if n==1
    output=1; %中止递归的条件
else
    output=n*fact(n-1);   %递归调用的语句
end
%调用时,输入fact(n)，n为自然数
```

§2.1.3 科契(Koch)雪花曲线

将等边三角形的
每条边三等分

将中间的一段去掉，代之以
一个更小的
等边三角形的两条边

依次做下去，将得到
图示的雪花分形图



1.分形雪花程序1

```
new=[0,i,sqrt(3)/2+i/2,0];
                          %初始的等边三角形，画图用4个数据
subplot(3,3,1);
plot(new),
axis equal
for kkk=1:8;                          %画图的次数
old=new;
n=length(old)-1;                      %计算线段的数目
```

（接下页）

```
for    jj=0:n-1;                                  %对每条线段进行计算
diff=(old(jj+2)-old(jj+1))/3;                     %线段1/3的长度
new(4*jj+2)=old(jj+1)+diff;                       %线段的1/3处
new(4*jj+3)=new(4*jj+2)+diff*((1+sqrt(3)*i)/2);
                                                  %转动后的点的坐标
new(4*jj+4)=old(jj+1)+2*diff;                     %线段的2/3处
end
new(4*n+1)=0;                                     %终点就是起点坐标
subplot(3,3,kkk+1),
plot(new),
axis equal;
end
```

2. 分形雪花程序2

```
new=[0,i,sqrt(3)/2+i/2,0];
    subplot(3,3,1);
    plot(new),
    axis equal;
    for k=1:8;
       old=new;
       n=length(old)-1;
diff = (old(2:end) - old(1:end-1)) / 3;
       new(1:4:4*n-3) = old(1:end-1);
```

（接下页）

（接上页）

```
new(2:4:4*n-2) = old(1:end-1) + diff;
new(3:4:4*n-1) = new(2:4:4*n-2) +
diff*(0.5+0.5*sqrt(3)*i);
      new(4:4:4*n) = old(1:end-1)+ diff + diff;
      new(4*n+1)=old(n+1);
      subplot(3,3,k+1);
      plot(new),
      axis equal;
    end
```

3. 分形雪花程序3

```
a=[0,i];                          %画初始三角形的一条边a
b=a*(-0.5 - 0.86603i)+i;
                                  %利用第一条的移动来画第二条边b
c=(a-i)*(-0.5 + 0.86603i);
                                  %利用第一条边的移动画第三条边c
subplot(3,3,1),
plot([a,b,c]);
axis equal
```

（接下页）

（接上页）

```
for k=1:8                                                    %画8个图
    subplot(3,3,k+1)
    a1=a/3;
        %将一条边缩小到1/3，并用它画新图第一条边a
    a=[a1,a1*(0.5 + 0.86603i)+i/3,(a1-i/3)*(0.5 -
0.86603i)+i*2/3, a1+2*i/3];
    b=a*(-0.5 - 0.86603i)+i;                                 %画第二条边b
    c=(a-i)*(-0.5 + 0.86603i);                               %画第三条边c
  plot([a,b,c])
  axis equal
  end
```

§2.1.4 L系统（阅读）

L系统 ⇌ 字符串重写系统

解释

- 字符串：曲线（基本图形）。
- 重写：按照一定的法则生成整个图形。

精髓

- 给一个起点（种子）和替换规则。
  连续利用规则生成新系统。

应用

- 生成规则比较明确的经典的分形图形。
- 模拟植物形态，特别是那些具有分枝结构的植物图形。

§2.2 复变函数迭代

§2.2.1 Julia集

$$Z_{k+1} = Z_k^{\,2} + C_0 \qquad (C_0 \neq 0)$$

画图方法

规定一个正的常数M和迭代次数N:

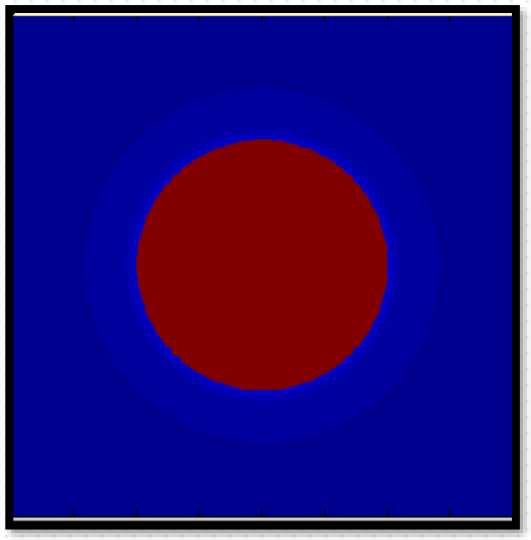
如果Z进行N次迭代后，仍然有$|Z| \leq M$，则该点属于Julia集，对于趋于内部的点也是如此处理。

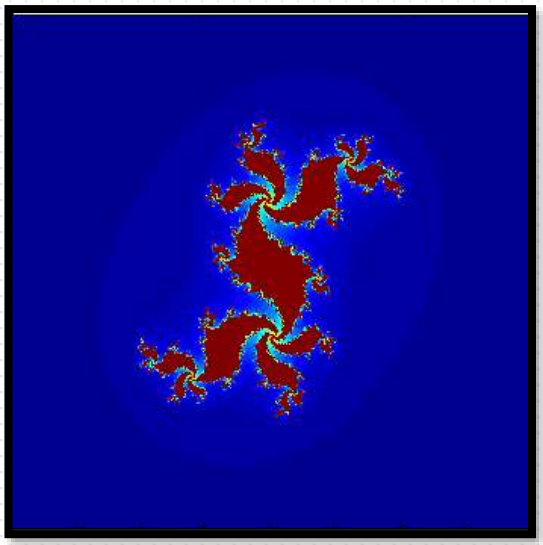
如果经过若干次迭代后，出现$|Z| > M$，则认为该点已经逃离，并根据达到逃离时需要的迭代次数对起始点进行分类，用不同的颜色来表示它们，就会得到非常美丽的图案。

1.画Julia set程序

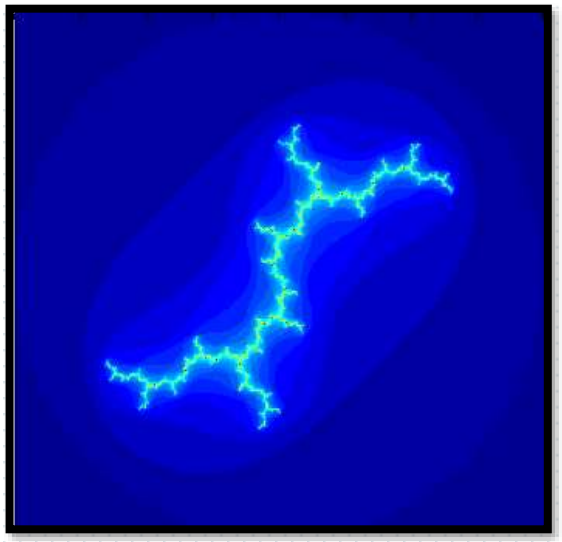
```
C=i;                                %迭代式中的常数，读者可以改变
V = linspace(-2.0,2.0,400);         %复数的范围及取点数
[Re,Im] = meshgrid(V);
Z = Re + i * Im;                    %复平面上所有起始点
B = 0;
for K = 1:100                       %迭代100次
    B = B+(abs(Z)<=2);              %已经逃离的判据
    Z = Z.*Z + C;
end;
imagesc(B);                         %表示成颜色图形
colormap(jet);
axis equal
```

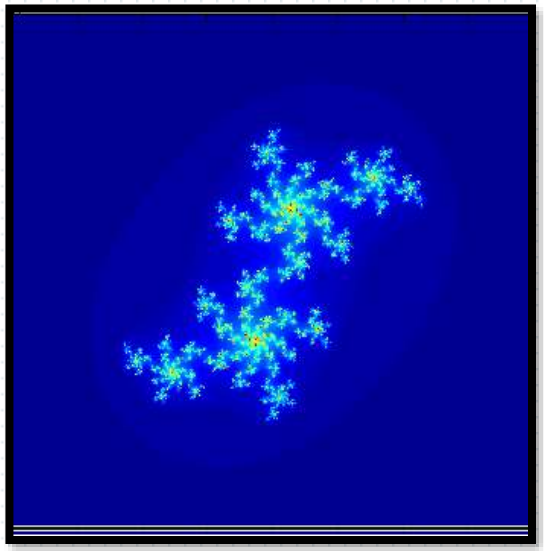
$C = 0$ $C = 0.3 + 0.5i$





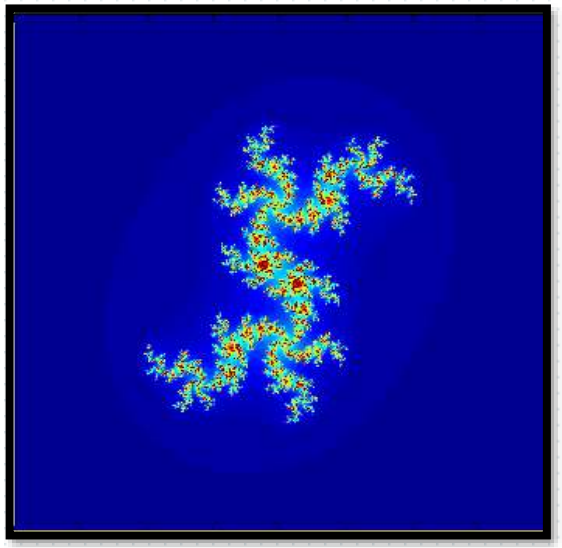
$C = i$ $C = 0.11 + 0.66i$



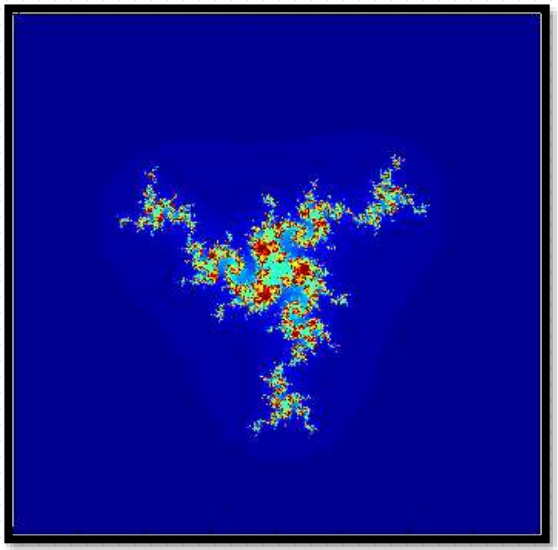


北京师范大学

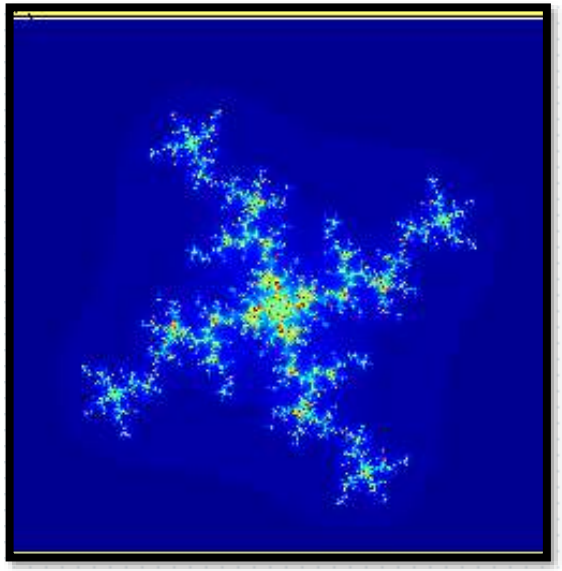
$Z = Z^2 + (0.32 + 0.5\mathrm{i})$



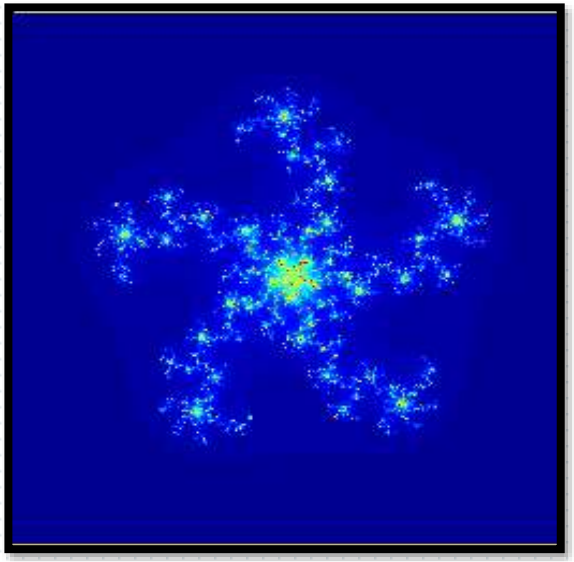
$Z = Z^3 + (0.2 + 1.05\mathrm{i})$



$Z = Z^4 + (0.4015 + 0.99i)$



$Z = Z^5 + (0.188 + 0.78603i)$



§2.2.2 Mandelbrot集

$$Z_0 = 0; \quad Z_{k+1} = Z_k^{\,2} + C_k$$

固定Z的起始点而改变C的取值进行迭代。

画图方法

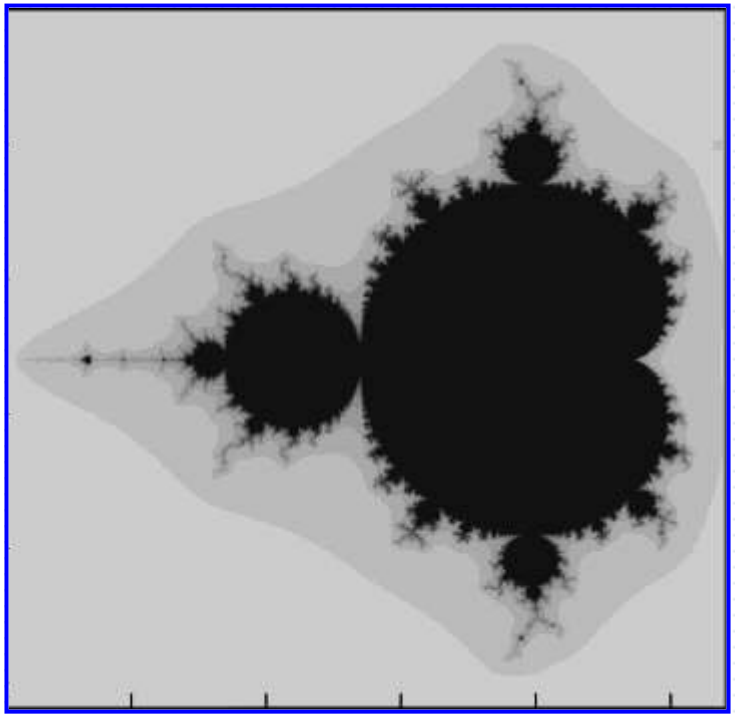
规定一个正常数M和迭代次数N：

如果经过N次迭代，仍然有$|Z| \le M$，C值就属于Mandelbrot集。

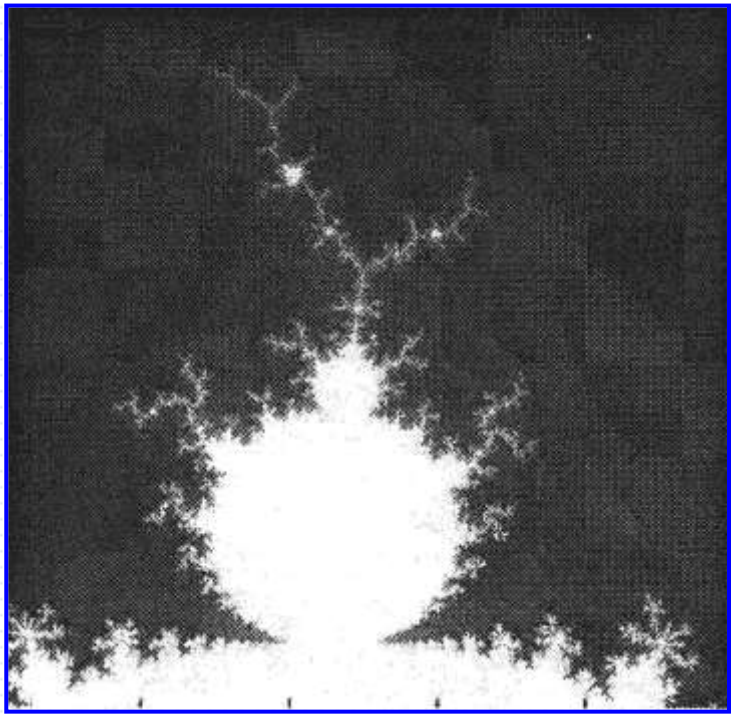
再根据达到逃离时所经过的迭代次数的不同对Mandelbrot集中的C加以分类并标示成不同的颜色就会得到分形图形。

Mandelbrot集图形（用程序界面演示）

整体图



局部放大图



北京师范大学

资料(视频)

http://www.kuqin.com/math/20110901/264161.html



§ 2.3 分形与分形维

分形

本意： 是不规则的、破碎的、分数的。

描述： 复杂无规的几何对象。

特点： 整体上看极不规则或极不光滑，但是整体与局部又存在某种相似。

定义： 曼德勃罗把部分与整体以某种方式相似的形体称为分形(fractal)。

分形维

分形理论用分形维数（简称分维）来定量描述分形结构自相似程度、不规则程度或破碎程度，它们的维数值不是整数。

1. 豪斯道夫维数

- 取长为l的1维线段，放大2倍后的长度$(2l)^1 = 2l$
- 边长为l的2维正方形，每边长放大2倍的面积为$(2l)^2 = 4l^2$
- 边长为l的3维立方体，每边长放大2倍的体积为$(2l)3 = 8l^3$

它们反映了维数的作用，结果整理如下：

一维图形（线段） $2^1 = 2$

二维图形（正方形） $2^2 = 4$

三维图形（立方体） $2^3 = 8$

归结出一般规律为：$L^D = K$

$D = 1,2,3$

取对数得豪斯道夫维数$D_f = \log K / \log L$

L对应于边长放大的倍数，K对应长度、面积、体积放大的倍数。

例：计算Sierpinski triangle的分维数

分析：

- 小三角形每边扩大2倍得大三角形
- 大三角形面积为小三角形4倍
- 减去中间小三角形，实际面积为小三角形3倍

计算：

$$D_f = \frac{\log K}{\log L} = \frac{\log 3}{\log 2} = 1.5849$$

结论：

- 得到谢尔宾斯基垫片的维数是1.5849而不是2，所以它不是二维图形

分维的概念很好地描述了Sierpinski triangle与欧几里得三角形之间的区别。

2.相似维数

假定某个几何体由N个局部组成，每个局部以相似比β与整体相似，则物体的相似维数为：

$$D_s = \frac{\log N}{\log(1/\beta)}$$

例 科契雪花曲线每条边由四个与整体相似的局部组成，每个局部的长度是整体长度1/3，即相似比$\beta = 1/3$，则有

$$D_s = \frac{\log N}{\log(1/\beta)} = \frac{\log 4}{\log 3} = 1.2618$$

3.盒维数

前两个定义：只适用于完全自相似的图形

盒　维　数：可用于不是完全自相似的图形

计算方法：

网孔数$N(s)$

将分析对象置于网格中，每个网孔的尺寸为s，数出包含了分析对象的网孔数目$N(s)$



网孔数$N(s)$

逐步缩小网孔的尺寸s，记录下对应的网孔数目$N(s)$



双对数曲线

画出$\log N(s)$和$\log(1/s)$图。如果它是直线，则它的斜率就代表分析对象的盒维数

例：英国海岸线的分维数

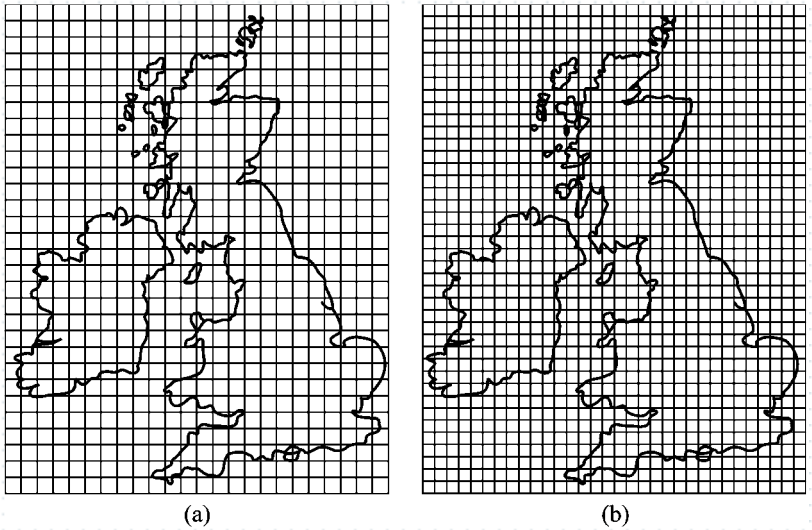
方法：将英国海岸线画在两个网格中，网格的总宽度都是1，网孔的尺寸分别为1/24和1/32，与海岸线相交的盒子数分别是194和283。

计算：双对数图中的直线斜率是

$$d=\frac{\log 283-\log 194}{\log 32-\log 24}\approx\frac{2.45-2.29}{1.51-1.38}\approx 1.31$$

结论：分维是一个不随系统的尺度改变而改变的量。

- 分形将尺度引入物理学。
- 分数维是一个系统的不变量。



(a) (b)



第二章完

北京师范大学